APPLICATION DEVELOPMENT PRACTICES

Dr.Lê Hùng Tiến

VLU

APPLICATION DEVELOPMENT PRACTICES

Analyzing & Estimating Requirements

Director’s Overview

- Requirements & Methodology
- Managing Traditional Requirements Change
- Managing Agile Requirements Change
- Estimating Change
- Using Change Management Tools

Traditional Methodologies

- Traditional methodologies assert the need for strong process discipline and rigorous practices
  - Quality products come from quality processes
  - **Assumes requirements are known at start**
  - **Assumes that a complete schedule & budget can be developed from the requirements**
  - Build products in stages
  - **Change is managed formally**
  - Rework = Waste

Agile Methodologies

- Agile methodologies use lighter, more adaptive paradigms
  - Product built so you always have a working system and can "release" at any time
  - Refactoring is good
- The Agile manifesto says that we have come to value:
  - Individuals & interaction over process & tools
  - Working software over comprehensive docs
  - **Customer collaboration over contract negotiation**
  - **Responding to change over following a plan**
  - That is, while there is value in the items on the right, we value the items on the left more

Requirements & Methodology

- Traditional
  - Big requirements up front
  - (Often) minimal user involvement
  - Big plan up front
  - **Control changes**
- Agile
  - Lightweight requirements
    - Emphasis is on working code
  - Must have user involvement
  - Minimal long range planning
  - **Change is embraced**

Managing Traditional Change

- Institute an attitude that change management is necessary
- Have a Documented Change Process
- Baseline Project Artifacts
- Build Rapport with Customer
- Work on prioritized change requests (CR) from a Change Control Board (CCB)
- Use Tools to Track Changes
  - Configuration Management – Manage revisions
  - Bug Tracker - Track change orders and their status
- Measure the changes
  - So you can quantify the impact of the changes
  - To improve your ability to estimate

An Attitude of Change – 1

- Set expectations with all stakeholders:
  - Change is inevitable
  - Let's agree on a change process so we all know how to work together to get the best results
  - Let's understand what the development processes are so we know what to expect
  - Other groups need to get involved to assess the change in terms of:
    - Test, Technical Publications, Support, etc.
  - Don't make changes until authorized to do so
    - The customer may not want the changes depending on the cost and schedule estimates

An Attitude of Change – 2

- Set expectations with all stakeholders:
  - Software Engineers will evaluate submitted changes in terms of:
    - Scope
    - Resources
    - Schedule
    - Cost
    - Risk
- Your estimate may not be what management submits to the customer
  - Don’t talk about money
  - Talk in ranges

Build Rapport with Customer

- We are doing this project to solve a customer/user problem or address a need
- Customer involvement and communications can
  - Reduce errors, misunderstandings and standoffs
  - Improve customer's perception and confidence in you and your project
- Work with the customer to:
  - Explain the software life cycle & its purpose
  - Provide accurate & trustworthy status & progress
  - Describe alternatives and their benefits and costs
  - Describe risks

Be Helpful and Supportive. It is their project

Problems Building Rapport with Customer – 1

- Customers often don't know what they want
  - Learn and use requirements-eliciting practices that help customers figure out what they want
  - Internal (to your company) customers:
    - Interface prototyping, Evolutionary prototyping, Joint Application Development (JAD), etc.
    - Conduct focus groups
  - External (to your company) customers:
    - Videotape people using the software
    - Conduct surveys to assess your relationship with your customers

Problems Building Rapport with Customer - 2

- Customers sometimes won’t commit to a set of written requirements
  - Possibly the requirements are too big, technical and hard to understand
  - Try describing the product in their language
    - Prototypes
    - Examples

Problems Building Rapport with Customer – 3

- Communication with customers can be slow
  - Communicate what you need and when with your customer, and what the impact is if you don't hear from them on time
  - If that doesn't work, then communicate with your customer that you need to let your management know that you are stuck with getting information from the customer
  - If that doesn't get your customer to respond, then escalate the issue to your management
  - Make sure you really need the customer's time
    - Don't make a big deal about something trivial

Problems Building Rapport with Customer – 4

- Customers will not (or can't) participate in reviews
  - Customers may not have the expertise
    - Use less technical, detailed or tedious specifications
    - Offer to provide some training
    - Offer encouragement
    - Look for another way
  - Customers may not have the time
    - Talk with your management about conveying to the customer the risk of project failure if the customer doesn't/can't participate

Problems Building Rapport with Customer – 5

- Customers won’t let people do their jobs
  - Help the customer focus on what, and not how
  - Help the customer understand the consequences of their technical requirements

Problems Building Rapport with Customer – 6

- Customers don’t understand the software development process
  - Don’t try to teach them about software lifecycles (unless they are interested)
  - Do explain to them the processes that they need to participate in, and their role in those processes
  - Keep them informed with what is coming up so they are aware and informed

Problems Building Rapport with Customer – 7

- Engineers often over promise and under deliver
  - Agreeing to unrealistic expectations will always damage your credibility
  - Use estimation techniques and let everyone know that you will be collecting data to improve your ability to estimate accurately
  - Never make promises about functionality, dates or cost without verifying your promises first

Problems Building Rapport with Customer - 8

- Engineers often don’t consult with their customers, especially if there is not a good relationship
  - Don’t let this happen!
  - Always treat the customer with respect
  - Always be professional, courteous and deferential
  - Try your best, even in difficult situations

Problems Building Rapport with Customer – 9

- Engineers often think they know better
  - Don’t add or modify features on your own (without CCB approval)
  - Don’t add or modify features without making sure the customer approves

Problems Building Rapport with Customer - 10

- Engineers build inflexible systems
  - Use good design practices to build systems that can be modified
  - Consider alternative designs and involve the customer with the final decision making

Traditional Requirements Change

- Must have baselined requirements
  - Else how will you know the scope of the new or changed requirement?
- Changes must only come from the CCB
  - Don't accept changes directly from end user, customer, senior management, etc.
  - How you can manage this successfully:
    - "Of course I will, but the project manager has said that we can only work on changes sent to us by the CCB. Can we run this by the project manager?"
    - "Of course, but shouldn't we follow the process?"
      - We need the CCB to do triage on the changes
      - We need to gather accurate change metrics

You May See A CR Multiple Times

- Analyze Change
  - You analyze a change to determine how long it will take, who will need to be involved, how much effort it will take, etc.
  - You pass this analysis information back to the CCB
- If the CCB (including the customer) agrees to implement the change, then you may be required to change the source code and coordinate any other changes with other groups

Avoid Common Problems

- Analyze changes quickly
  - Do a few well rather than many poorly
- Involve stakeholders in the change process
- Use discipline when following the change process
- Follow a checklist when analyzing changes
  - Who else needs to be involved?
  - What work needs to be done for the change to be completed?
- Collect and analyze change data
- Keep good design notes
  - You may not be who implements the change

Agile Change Management – 1

- Fourth "value" of the Agile Manifesto
  - "Responding to change over following a plan"
  - Change in mindset:
    - Goal isn't to prevent changes to an agreed to baseline
    - Goal is to embrace change as a natural part of development
- Customers/Users (Product Owner)
  - Prioritize the 'user stories' on the 'product backlog' to be implemented at the beginning of each iteration/sprint (see 'planning game')
  - Are available to help answer the developer's implementation questions

Source: http://www.cmcrossroads.com/articles/agileaug03.html

Agile Change Management – 2

- Agile methods enable projects to be responsive to change by:
  - Using short and frequent iterations to minimize the time between specifying a requirement & seeing it implemented
  - Requiring developers and customers to communicate and work together daily (co-located together if possible)
  - Accommodating (even encouraging) changes in scope and priorities by informing the customer of impact and risk, & putting the customer in control of the scope & priorities

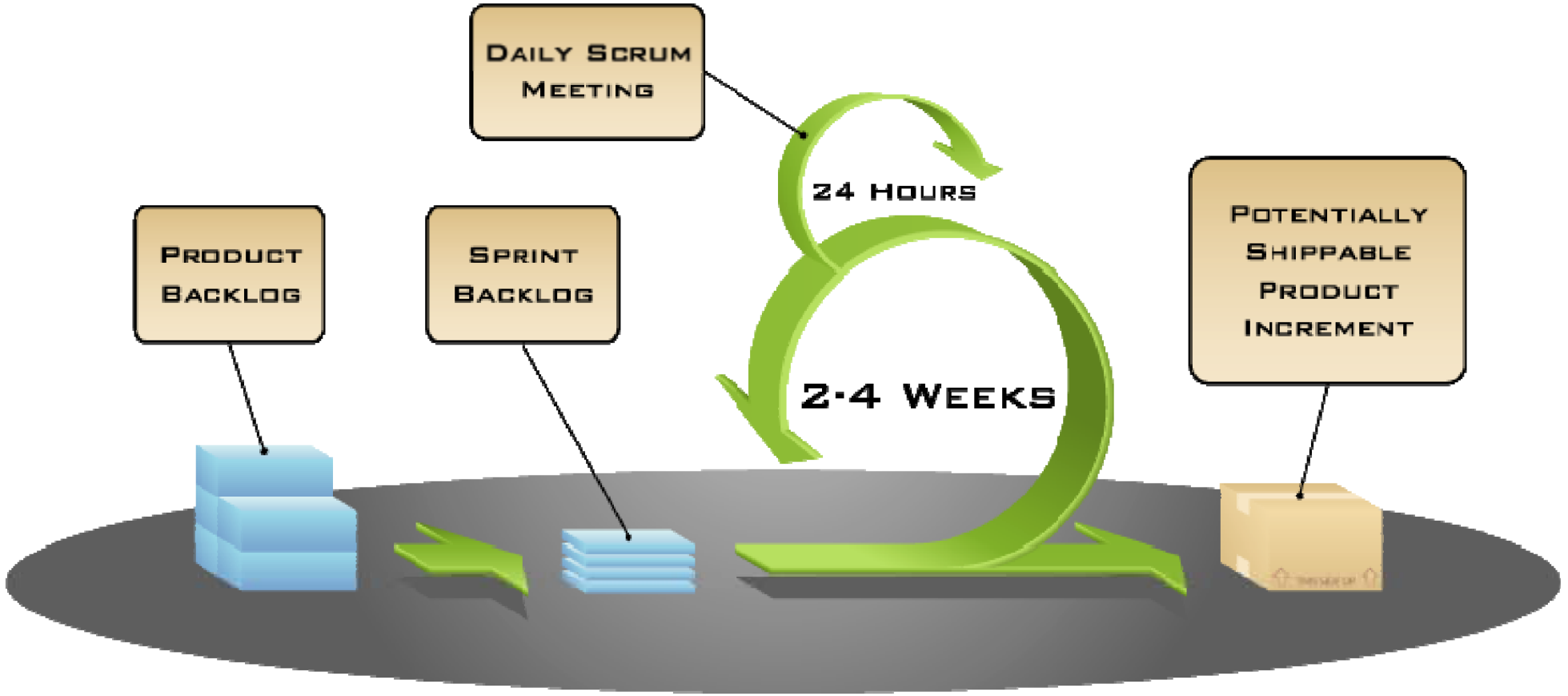
Source: http://www.cmcrossroads.com/articles/agileaug03.html

Agile Change Management – 3

- Agile methods enable projects to be responsive to change by:
  - Authorizing and empowering team members to correct problems with the code's behavior and structure without having to suffer delays waiting for formal change process
- Little or no change tracking or traceability with Agile methods
  - Focus on keeping things as simple as possible

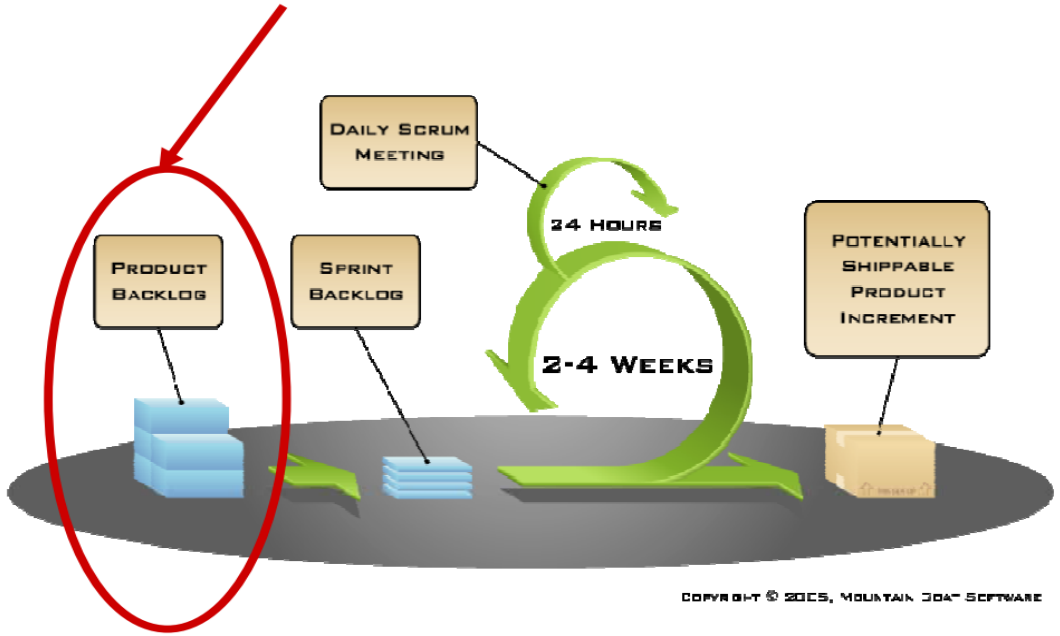
Source: http://www.cmcrossroads.com/articles/agileaug03.html

Types of Agile Change



1: Product Backlog

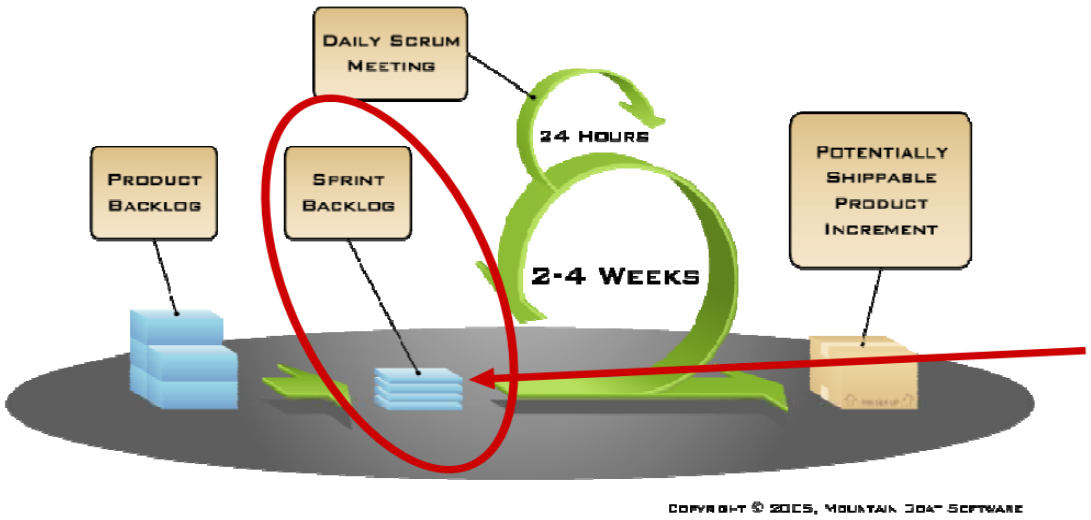
- The work to be done on a Scrum project is listed on a "product backlog"



- A list of all project work, including requirements
- Prioritized by the product owner
- Reprioritized by the product owner during planning at the start of each sprint

2: Sprint Backlog

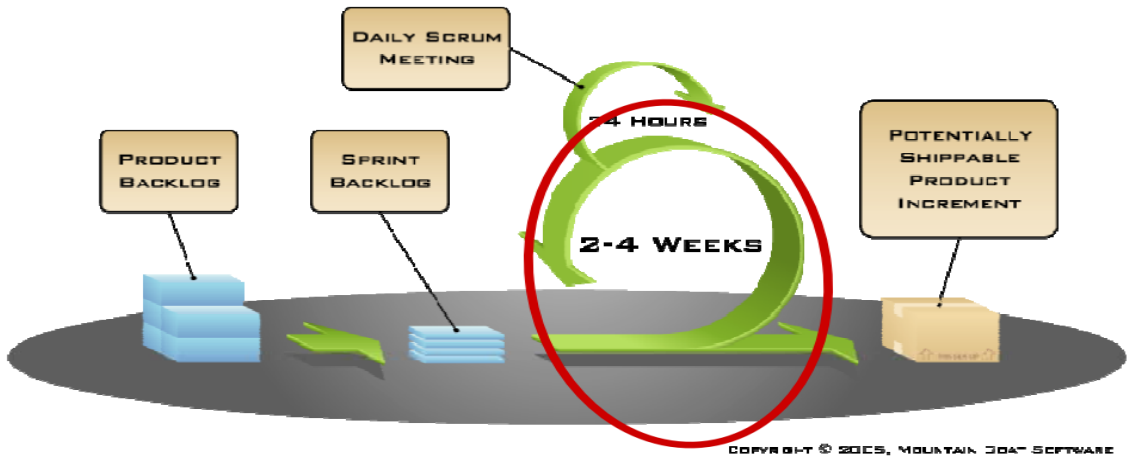
- At the start of each sprint a Sprint Planning Meeting is held



- The Product Owner prioritizes the Product Backlog
- The Scrum Team selects the tasks they can complete during the coming Sprint
- These tasks are then moved from the Product Backlog to the Sprint Backlog

3: During the Sprint

- Development is happening



- The Product Owner is available to provide clarity to developers on what is desired in terms of feature look, feel & implementation
- User stories may be broken into other stories or sub-stories
- The Product Owner can not add new stories in the middle of a sprint

Use Change Management Tools

- CR Tracking system
  - Fully document the change
- Configuration Management
  - Check in one change at a time
- Log other data, as required
  - Effort

Change Estimation ...

- Is the same for each methodology as estimating original requirements:
- Traditional:
  - Use a work breakdown structure (wbs) to consider all work
  - Use historical data, parametric or wideband delphi estimation method
- Agile:
  - Estimate available hours for the iteration
  - Repeat until full:
    - Pick a story, break into tasks, estimate each task
  - Track and use historical data to refine velocity

Summary – Traditional

- You need to implement important customer changes, but too many changes can kill the project
- Your change management process can help filter out ‘non-essential’ changes, however you risk losing customer rapport if your change management system is perceived as unwieldy and bureaucratic
- Communicate openly and honestly with your stakeholders to find the right balance for your project

Summary – Agile

- Change is a natural part of the agile development process
- You need to implement important customer changes, but too many changes can kill the project
- It is up to the product owner to filter out ‘non-essential’ changes
- Communicate openly and honestly with your stakeholders to find the right balance for your project